Portfolio Assessment 3
Advanced ML and Deep Learning

Pham Quang Minh | Student ID: 104999568

COS40007 Artificial Intelligence Engineering - Portfolio Assessment 3

Date: 2 November 2026 | Swinburne University of Technology

Option 3d: Classification with a tabular ANN

GitHub: https://github.com/theh9j/COS40007

1. Problem and data preparation

The aim is to predict a concrete sample's relative strength class using its ingredients and age. The four classes are Lower, Lower-middle, Upper-middle and Higher. They continue Portfolio 2 and are **relative dataset groups, not engineering safety grades**.

GenAI declaration

OpenAI Codex (GPT-6) assisted with debugging, code polishing and the report. The overall structures, interpretations and final say are my own.

What was kept consistent

Item	Preparation
Data	1,030 source records; 25 duplicate copies removed in earlier preparation; 1,005 retained. No missing values.
Inputs	Cement, slag, fly ash, water, superplasticizer, coarse aggregate, fine aggregate and age. Strength, labels and IDs excluded.
Target	Fixed Portfolio 2 cutoffs: 23.2393, 33.4542 and 44.3436 MPa (rounded here). The notebook uses the full saved values.

Outer split	676 training rows and 329 test rows. Samples with the same ingredient recipe stay together, including samples measured at different ages.
Scaling	StandardScaler learns only from fitting rows, including separately inside each traditional CV fold.

Keeping related recipes together reduces leakage: otherwise a model could see almost the same mixture during training and testing. The saved outer split is reused for every model. Earlier portfolios already explored this dataset, so this test set is a comparison set rather than independent external validation.

How performance is measured

Accuracy is the fraction of correct class predictions. Weighted F1 balances precision and recall while weighting each class by its number of records. Five-fold recipe-group cross-validation (CV) is used for the two traditional models. The ANN uses a separate validation set; its validation score is not a CV score.

2. Traditional model baselines

Random Forest uses 200 trees and at least two records per leaf. The scikit-learn MLP uses hidden layers of 100 and 50 units, ReLU, Adam and a 500-iteration limit. Both use the same five recipe-group folds and scaler pipelines.

Model	Mean CV accuracy	Test accuracy	Test weighted F1
Random Forest	69.2%	78.1%	0.781
MLP	75.2%	82.1%	0.820

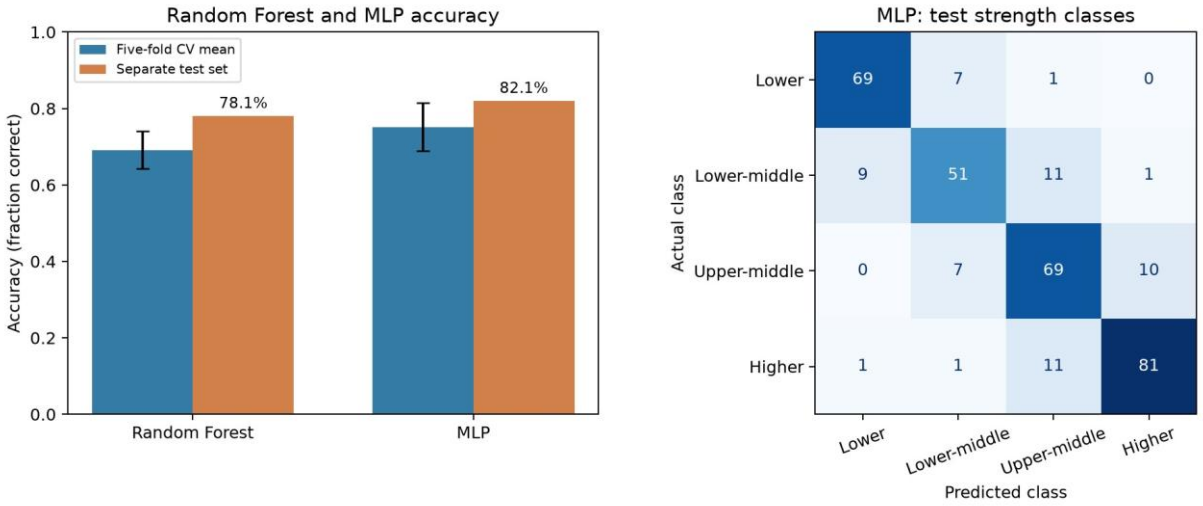


Figure 1. Traditional model comparison and MLP confusion matrix. CV error bars show fold standard deviations, not confidence intervals.

Interpretation

The MLP is the traditional baseline selected by mean CV accuracy. Its test accuracy is also higher than the Random Forest's. Both models achieve about 97% training accuracy, well above CV performance, so their training scores are optimistic.

Limitation: all six MLP fits (five folds and the final fit) reached 500 iterations without convergence. Its score is useful as a fixed baseline, but it should not be described as fully converged. Both complete scaler/model pipelines are saved as pickle files.

3. ANN design and first experiment

A dense ANN suits these eight tabular inputs. The age column does not make the independent samples a time series, so an LSTM is unnecessary. Four softmax outputs represent the four classes.

Layer	Output units	Parameters
Dense + ReLU	64	576
Dropout (0.2)	64	0
Dense + ReLU	32	2,080
Dropout (0.2)	32	0
Dense + ReLU	16	528
Dense + softmax	4	68
Total trainable parameters		3,252

From the 676 training rows, 20% of recipe groups are reserved for validation: **513 fitting rows and 163 validation rows**. Group sizes vary, so the row percentage differs from 20%. The ANN scaler fits only those 513 rows.

Settings: Adam learning rate 0.001, sparse categorical cross-entropy, batch size 32 and seed 42. Early stopping monitors validation loss with patience 10 and restores the best weights. The learning rate halves after five unimproved epochs. Saved checkpoints select the lowest validation loss.

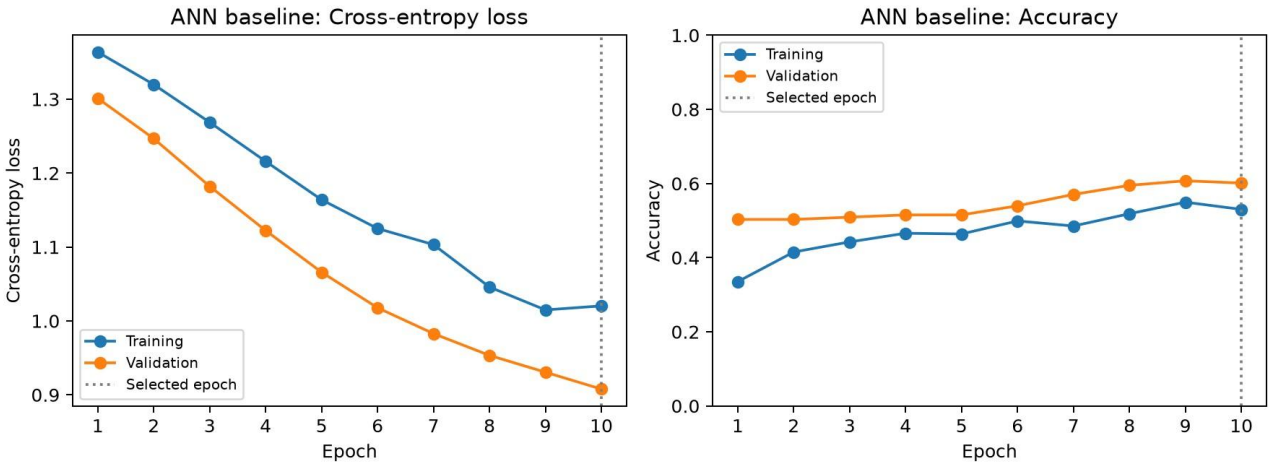


Figure 2. Training and validation loss/accuracy for the 10-epoch ANN.

All 10 epochs completed, with the best validation loss at epoch 10. Test accuracy was **62.3%** and weighted F1 **0.583**. Loss was still improving, motivating the already planned longer run. Dropout is active during training but off during validation, so validation accuracy can exceed the training curve.

4. Controlled longer-training experiment

Only the maximum epoch budget changed: **10 to 50**. The model restarted from the same seed; it did not continue from the first model's trained weights. Layers, scaler, recipe split, batch size and callbacks stayed fixed. The first ten epochs reproduce the original history.

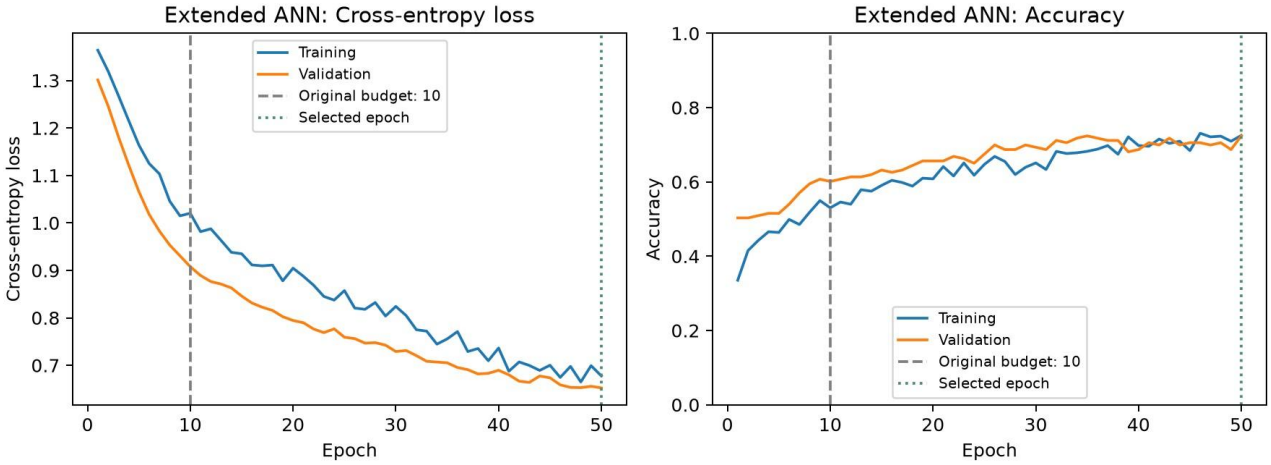


Figure 3. Extended learning curves. The dashed line marks the original budget; the selected weights have the lowest validation loss.

Measure	10-epoch ANN	50-epoch ANN
Selected / completed epoch	10 / 10	50 / 50

Validation accuracy	60.1%	72.4%
Validation loss	0.908	0.652
Test accuracy	62.3%	75.7%
Test weighted F1	0.583	0.746
Test loss	0.932	0.602

The longer run is the preferred ANN because its **validation loss is lower**, not because its test score was used for selection. Its test accuracy increased by 13.4 percentage points. All 50 epochs completed; early stopping did not activate before the limit. Reaching the limit does not prove convergence.

Both ANN variants are saved in H5 format with native Keras checkpoints, the fitting-only scaler and pickled/CSV histories. Reload checks reproduced their predictions. No additional settings were tried after inspecting these test results.

5. Final comparison and conclusion

Model	Test accuracy	Weighted F1	Mean CV accuracy
Random Forest	78.1%	0.781	69.2%
MLP	82.1%	0.820	75.2%
ANN (10 epochs)	62.3%	0.583	Not run
ANN (up to 50 epochs)	75.7%	0.746	Not run

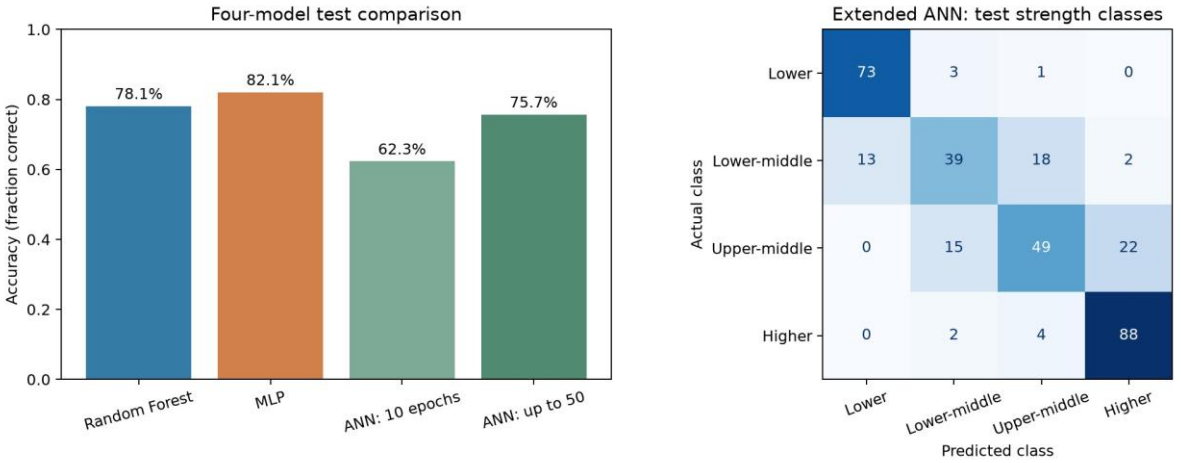


Figure 4. Test accuracy for all four models and the extended ANN confusion matrix. All models use the same 329 test rows.

Main finding: the MLP has the highest observed test accuracy (82.1%). More training improved the Keras ANN, but it did not exceed either traditional baseline in this experiment.

The extended ANN recognises the extreme classes better: recall is 94.8% for Lower and 93.6% for Higher, compared with 54.2% and 57.0% for the two middle classes. The confusion matrix shows where its predictions need improvement.

Limits and next step: the ANN fits 513 rows while traditional models fit 676, so this is not an isolated architecture comparison. Results come from one reused test set, one ANN seed and two planned budgets. Keep the MLP convergence warning visible. A future study could compare matched fitting sets, use grouped validation for tuning, repeat seeds, then assess a fresh external set. These class predictions are not a structural-safety assessment.

6. GenAI reflection and references

Tool and assistance record

Tool: OpenAI Codex desktop coding agent. The following prompts were used during the completion of the assignment:

- "Provide me suggestion on how to set up an ANN model"
- "Take a look at this program and let me know if it's correct or if there are any issues with it"
- "I've written the general layout for the report, please help me continue with the improvement"

Personal learning reflection

During this project, setting up and structuring the ANN model was the most tedious and difficult part because I had never attempted it before.

References and attribution

Yeh, I. (1998). Concrete Compressive Strength. UCI Machine Learning Repository. DOI: 10.24432/C5PK67. Data attribution retained from the earlier portfolios; CC BY 4.0. Prepared labels and recipe partitions continue Portfolio 2.

Course materials: AssignmentTasks.pdf; Portfolio Task 3d - Advanced ML Models and Deep Learning - MLP for Classification.pdf; MLP_Classifcation.ipynb; ANN_Regression_Classification.ipynb; DL_Models_ExampleCode.ipynb. Teaching examples were adapted to the eight concrete inputs and four classes.

Implementation references: scikit-learn RandomForestClassifier, MLPClassifier and grouped cross-validation APIs; Keras Sequential, Dense, Dropout and callback APIs.

Project: https://github.com/theh9j/COS40007.